

Python爬蟲入門-Selenium

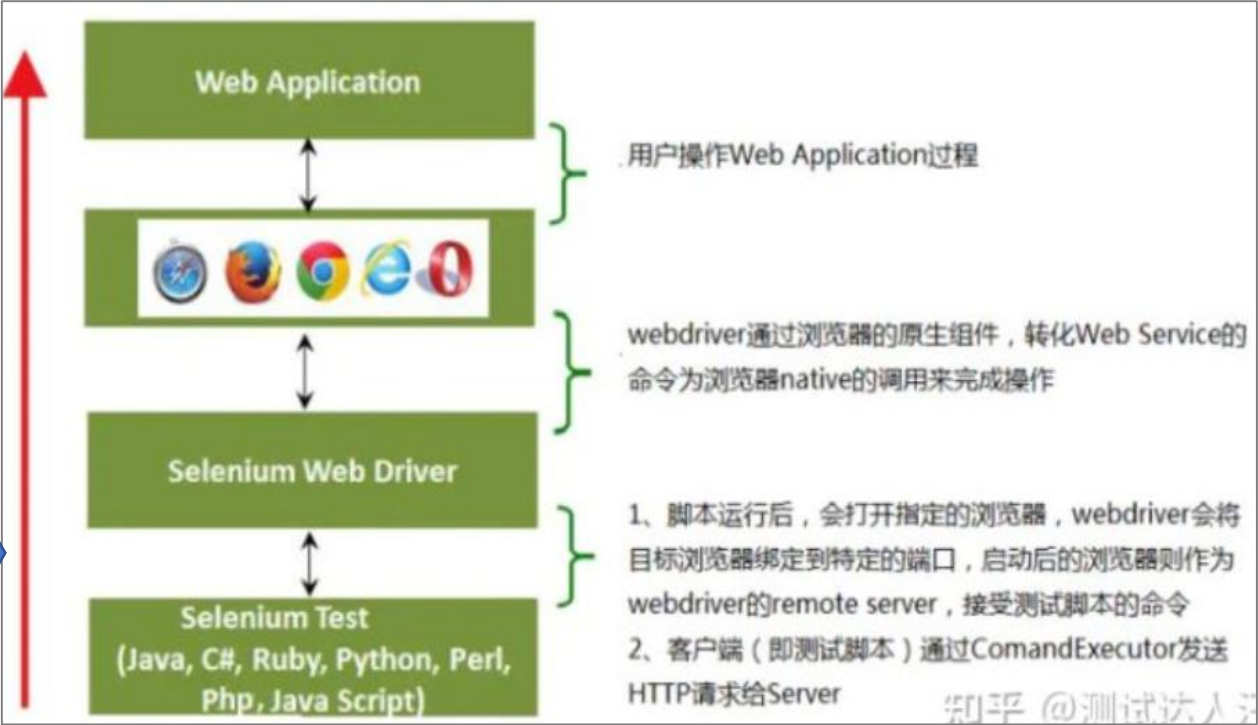
By Wendy Chin

On Feb., 15th, 2023

Selenium簡介

- Selenium是一个用于Web应用程序测试的工具。只要在测试用例中把预期的用户行为与结果都描述出来，就得到了一个可以自动化运行的功能测试套件。
- Selenium测试套件直接运行在浏览器中，就像真正的用户在操作浏览器一样。
- 其中WebDriver 從Selenium 2.0開發至今，對原始瀏覽器直接驅動，是目前使用最方便的工具之一

工具	描述
Selenium IDE	Selenium 集成开发环境（IDE）是一个Firefox插件，可以让测试人员跟着，需要测试的工作流程，以记录他们的行为。
Selenium RC	Selenium远程控制（RC）为旗舰测试框架，它允许多个简单的浏览器动作和线性执行。它使用的编程语言，如Java, C #, PHP, Python和Ruby和Perl的强大功能来创建更复杂的测试。
Selenium WebDriver	Selenium的webdriver前身是Selenium RC，直接发送命令给浏览器，并检索结果。
Selenium Grid	Selenium网格用于运行在不同的机器，不同的浏览器同时以最小化执行时间的并行测试的工具。



檢查瀏覽器版本-Chrome



按照順序分別點擊Step1~Step3

下載WebDriver-Chrome

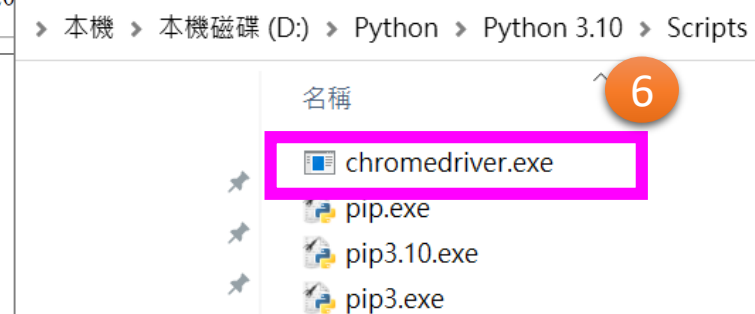
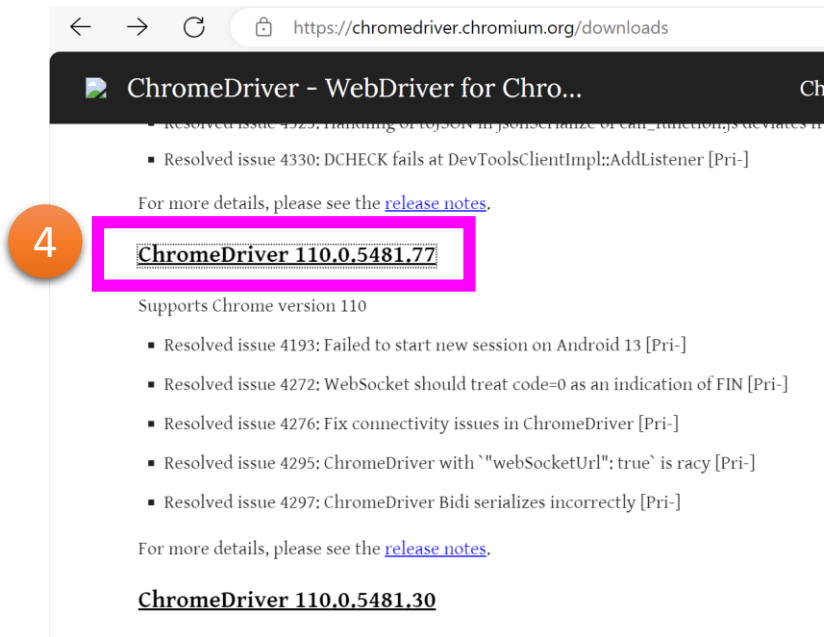
Google Chrome驅動下載：<https://chromedriver.chromium.org/downloads>

或 <https://googlechromelabs.github.io/chrome-for-testing/>

MS Edge驅動下載：<https://developer.microsoft.com/en-us/microsoft-edge/tools/webdriver/>

MS IE驅動下載：<http://selenium-release.storage.googleapis.com/index.html>

- 按照順序分別點擊Step 4, Step5, 下載保存到個人電腦本地盤



- 下載的文件解壓縮后將*.exe文件放入到Python安裝路徑下的<Scripts>文件夾中。沒有下載權限可直接到FTP中查找

檢查瀏覽器版本-MS Edge

1

...

新索引標籤Ctrl+T

新視窗Ctrl+N

新增 InPrivate 視窗Ctrl+Shift+N

縮放— 100% + ↗

我的最愛Ctrl+Shift+O

歷程記錄Ctrl+H

購物

下載Ctrl+J

應用程式>

擴充功能

瀏覽器基本功能

列印Ctrl+P

網頁擷取Ctrl+Shift+S

在頁面上尋找Ctrl+F

更多工具>

2

設定

說明與意見反應

關閉 Microsoft Edge

由您的組織管理

Microsoft Edge WebDriver

Close the loop on your developer cycle by automating testing of your website in Microsoft Edge with Microsoft Edge WebDriver

Navigate to installation and usage >

4

關於 Microsoft Edge

版本 120.0.2210.144 (官方組建) (64 位元)

Microsoft Edge 已更新至最新。

說明F1

傳送意見反應Alt+Shift+I

報告不安全的網站

新增功能與提示

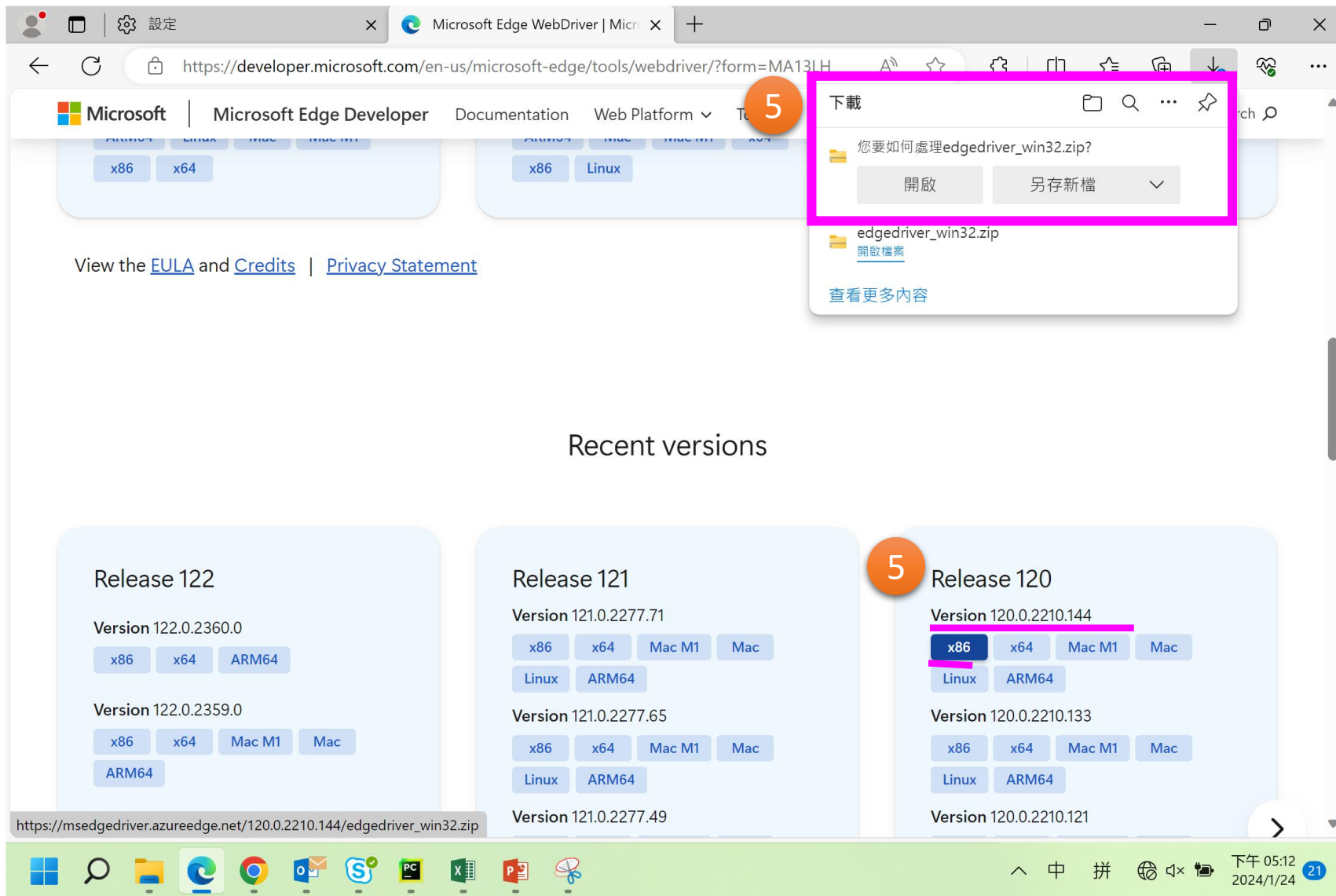
3

關於 Microsoft Edge

Get the latest version

下午 07:46 2023/8/1

下載WebDriver-MS Edge



- 下載的文件解壓縮后將*.exe文件放入到Python安裝路徑下的<Scripts>文件夾中。沒有下載權限可直接到FTP中查找



安裝Selenium資源包

- 按照順序Step1~15分別離綫安裝資源包，資源包可至FTP中獲取

Selenium_list.txt - 記事本

檔案(F) 編輯(E) 格式(O) 檢視(V) 說明(H)

```
D:\Python\Python312_KU\Selenium41717\urllib3-2.1.0-py3-none-any.whl
D:\Python\Python312_KU\Selenium417\attrs-23.2.0-py3-none-any.whl
D:\Python\Python312_KU\Selenium417\sortedcontainers-2.4.0-py2.py3-none-any.whl
D:\Python\Python312_KU\Selenium417\idna-3.6-py3-none-any.whl
D:\Python\Python312_KU\Selenium417\outcome-1.3.0.post0-py2.py3-none-any.whl
D:\Python\Python312_KU\Selenium417\sniffio-1.3.0-py3-none-any.whl
D:\Python\Python312_KU\Selenium417\pyparser-2.21-py2.py3-none-any.whl
D:\Python\Python312_KU\Selenium417\cffi-1.16.0-cp312-cp312-win_amd64.whl
D:\Python\Python312_KU\Selenium417\trio-0.24.0-py3-none-any.whl
D:\Python\Python312_KU\Selenium417\h11-0.14.0-py3-none-any.whl
D:\Python\Python312_KU\Selenium417\wsproto-1.2.0-py3-none-any.whl
D:\Python\Python312_KU\Selenium417\trio_websocket-0.11.1-py3-none-any.whl
D:\Python\Python312_KU\Selenium417\certifi-2023.11.17-py3-none-any.whl
D:\Python\Python312_KU\Selenium417\pysocks-1.7.1-py3-none-any.whl
D:\Python\Python312_KU\Selenium417\selenium-4.17.2-py3-none-any.whl
```

```
import os

path="C:\\Users\\wendy_chin\\Desktop\\Demo\\"
flnm="Selenium_list.txt"

fl=open(path+flnm,"r")      # r以只读方式打开
list1=fl.readlines()       #逐行读取
fl.close()

for ls in list1:
    ls1=ls.rstrip('\n')      #去掉换行符
    os.system('pip install '+ls1)
```

Tips:
也可用左側與
project同路徑
下.py文件的代
碼運行后，批
量安裝15個資源包。

- Settings中檢查安裝情況。
- 安裝成功畫面如下：

Settings

Project: Python_Test > Python Interpreter

Python Interpreter: Python 3.10 (venv) (5) D:\Pyt

Package	Version
pyparser	2.21
pyparsing	3.0.7
python-dateutil	2.8.2
python-docx	0.8.10
python-pptx	0.6.21
pytz	2021.3
pywin32	303
schedule	1.1.0
selenium	4.7.2
setuptools	58.1.0

Selenium常用方法屬性-元素定位與返回元素信息

设browser1=webdriver.Chrome(service=ser)		
定位一个元素	定位多个元素	含义
find_element(By.ID,"x")	find_elements(By.ID,"x")	通过元素id定位
find_element(By.NAME,"x")	find_elements(By.NAME,"x")	通过元素name定位
find_element(By.XPATH,"x")	find_elements(By.XPATH,"x")	通过xpath表达式定位
find_element(By.LINK_TEXT,"x")	find_elements(By.LINK_TEXT,"x")	通过完整超链接定位
find_element(By.PARTIAL_LINK_TEXT,"x")	find_elements(By.PARTIAL_LINK_TEXT,"x")	通过部分链接定位
find_element(By.TAG_NAME,"x")	find_elements(By.TAG_NAME,"x")	通过标签定位
find_element(By.CLASS_NAME,"x")	find_elements(By.CLASS_NAME,"x")	通过类名进行定位
find_element(By.CSS_SELECTOR,"x")	find_elements(By.CSS_SELECTOR,"x")	通过css选择器进行定位

获取断言信息：不管是在做功能测试还是自动化测试，最后一步需要拿实际结果与预期进行比较。这个比较的称之为断言	
属性	说明
title	用于获得当前页面的标题
current_url	用户获得当前页面的URL
text	获取搜索条目的文本信息

Selenium常用方法屬性-瀏覽器控制與特殊鍵盤控制

webdriver模块常用方法的使用：	
方法	说明
set_window_size()	设置浏览器的大小
maximize_window()	設置瀏覽器最大化
minimize_window()	設置瀏覽器最小化
back()	控制浏览器后退
forward()	控制浏览器前进
refresh()	刷新当前页面
clear()	清除文本
send_keys (value)	模拟按键输入
click()	单击元素
submit()	用于提交表单
get_attribute(name)	获取元素属性值
is_displayed()	设置该元素是否用户可见
size	返回元素的尺寸
text	获取元素的文本

窗口截图	
方法	说明
get_screenshot_as_file(self, filename)	用于截取当前窗口，并把图片保存到本地

特殊键盘事件	
模拟键盘按键	说明
send_keys(Keys.BACK_SPACE)	删除键 (BackSpace)
send_keys(Keys.SPACE)	空格键(Space)
send_keys(Keys.TAB)	制表键(Tab)
send_keys(Keys.ESCAPE)	回退键 (Esc)
send_keys(Keys.ENTER)	回车键 (Enter)
send_keys(Keys.CONTROL, 'a')	全选 (Ctrl+A)
send_keys(Keys.CONTROL, 'c')	复制 (Ctrl+C)
send_keys(Keys.CONTROL, 'x')	剪切 (Ctrl+X)
send_keys(Keys.CONTROL, 'v')	粘贴 (Ctrl+V)
send_keys(Keys.F1...Fn)	键盘 F1...Fn

多窗口切换	
方法	说明
current_window_handle	获得当前窗口句柄
window_handles	返回所有窗口的句柄到当前会话
switch_to.window()	用于切换到相应的窗口，与上一节的switch_to.frame()类似，前者用于不同窗口的切换，后者用于不同表单之间的切换。

关闭浏览器	
方法	说明
close()	关闭单个窗口
quit()	关闭所有窗口

Selenium常用方法屬性-鼠標控制

WebDriver 中，將這些關於鼠標操作的方法封裝在 ActionChains 類提供		
方法	說明	範例
ActionChains(driver)	構造ActionChains對象	<pre>from selenium.webdriver.common.action_chains import ActionChains Elem1=browser1.find_element_by_link_text('設置') ActionChains(browser1).move_to_element(Elem1).perform()</pre>
context_click(element)	用於模擬鼠標右鍵操作，在調用時需要指定元素定位	
move_to_element(element)	執行鼠標懸停操作	
double_click(element)	双击	<pre>Elem1=browser1.find_element_by_link_text('設置') Elem1.double_click()</pre>
drag_and_drop(elem1,elem2)	拖动,從elem1拖到elem2	
move_by_offset(xoffset, yoffset)	移动鼠标到指定的x, y位置（相对于浏览器的绝对位置）	
move_to_element_with_offset(element, xoffset, yoffset)	相对element元素,移动鼠标到指定的x, y位置（相对于element元素的相对位置）	
click_and_hold(element1=None)	在element1元素上按下鼠标左键，并保持按下动作（元素默认为空）	
release(element2=None)	在element2元素上松开鼠标左键（元素默认为空）	
key_down(key, element1=None)	在element1元素上，按下指定的键盘key（ctrl、shift等）键，并保持按下动作（元素默认为空）	
key_up(key, element2=None)	在element2元素上，松开指定的键盘key（元素默认为空）	
send_keys(key)	向当前定位元素发送某个key键	
send_keys_to_element(element, key)	向element元素发送某个key键	
pause(n)	暫停n秒	<pre>ActionChains(browser1).move_to_element(Elem1).pause(2).perform()</pre>
scroll_to_element(element)	滾動到某個元素，僅用於Chrome	
scroll_from_origin(element,x,y)	從一個元素滾動到指定位置(x,y),僅用於Chrome	<pre>scrollorigin=ScrollOrgin.from_element(element, x0,y0) ActionChains(browser1).scroll_from_origin(scrollorigin,x,y).perform()</pre>
perform()	執行所有 ActionChains 中存儲的行為，可以理解成是對整個操作的提交動作	

Selenium常用方法屬性-警告框、下拉框、上傳、iframe

警告框处理：在WebDriver中处理JavaScript所生成的alert、confirm以及prompt十分简单，具体做法是使用 switch_to.alert 方法定位到 alert/confirm/prompt，然

方法	说明	範例
text	返回 alert/confirm/prompt 中的文字信息	alert=browser1.switch_to.alert tx=alert.text
accept()	接受现有警告框	alert=browser1.switch_to.alert alert.accept()
dismiss()	解散现有警告框	alert=browser1.switch_to.alert alert.dismiss()
send_keys(keysToSend)	发送文本至警告框。keysToSend：将文本发送至警告框。	alert=browser1.switch_to.alert alert.send_keys('userID')

下拉框选择操作,from selenium.webdriver.support.select import Select

方法	说明	範例
select_by_value("选择值")	select标签的value属性的值	Select(element).select_by_value('20')
select_by_index("索引值")	下拉框的索引	Select(element).select_by_index(6)
select_by_visible_text("文本值")	下拉框的文本值	

文件上传

对于通过input标签实现的上传功能，可以将其看作是一个输入框，即通过send_keys()指定本地文件路径的方式实现文件上传。

browser1.find_element_by_name('file').send_keys("D:\\Demo.jpg")

在Web应用中经常会遇到frame/iframe表单嵌套页面的应用，WebDriver只能在一个页面上对元素识别与定位

方法	说明
switch_to.frame()	将当前定位的主体切换为frame/iframe表单的内嵌页面中
switch_to.default_content()	跳回最外层的页面

Selenium: 常用模組、啟動WebDriver (Chrome)

```
1 from selenium import webdriver
2 from selenium.webdriver.chrome.service import Service
3 from selenium.webdriver.common.by import By
4 import time
5 from selenium.webdriver.support.select import Select
6 from selenium.webdriver.support.ui import WebDriverWait
7 from selenium.webdriver.common.action_chains import ActionChains
8 from selenium.webdriver.common.keys import Keys
9 import datetime
10 import pandas as pd
11 # 因selenium版本升级后，须导入webdriver下的Service模組，否则会报DeprecationWarning的错误(实际上不影响运行)，
12 # 是之前的 executable_path 被重构到了 Service 函数里
13 # DeprecationWarning: executable_path has been deprecated, please pass in a Service object
14
15 #不自動關閉瀏覽器
16 option=webdriver.ChromeOptions()
17 option.add_experimental_option('detach', True)
18
19 driverpath='D:\\Python\\Python310\\Scripts\\chromedriver.exe'
20 ser=Service(driverpath)
21 browser1=webdriver.Chrome(service=ser, options=option)
22 url1='http://fip.pegatroncorp.com/FIP2/'
23 browser1.get(url1)
24 time.sleep(2.0)
```

兩者需一致

Selenium: 常用模組、啟動WebDriver (MS Edge)

```
1 from selenium import webdriver
2 from selenium.webdriver.edge.service import Service
3 # from selenium.webdriver.chrome.service import Service
4 from selenium.webdriver.common.by import By
5 import time
6 from selenium.webdriver.support.select import Select
7 from selenium.webdriver.support.ui import WebDriverWait
8 from selenium.webdriver.common.action_chains import ActionChains
9 from selenium.webdriver.common.keys import Keys
10 import datetime
11 import pandas as pd
12 # 因selenium版本升级后，须导入webdriver下的Service模组，否则会报DeprecationWarning的错误(实际上不影响运行)，
13 # 是之前的 executable_path 被重构到了 Service 函数里
14 # DeprecationWarning: executable_path has been deprecated, please pass in a Service object
15
16 #不自動關閉瀏覽器
17 option=webdriver.EdgeOptions()
18 option.add_experimental_option('detach', True)
19
20 driverpath='D:\\Python\\Python310\\Scripts\\msedgedriver.exe'
21 ser=Service(driverpath)
22 browser1=webdriver.Edge(service=ser, options=option)
23 url1='http://fip.pegatroncorp.com/FIP2/'
24 browser1.get(url1)
25 time.sleep(2.0)
```

兩者需一致

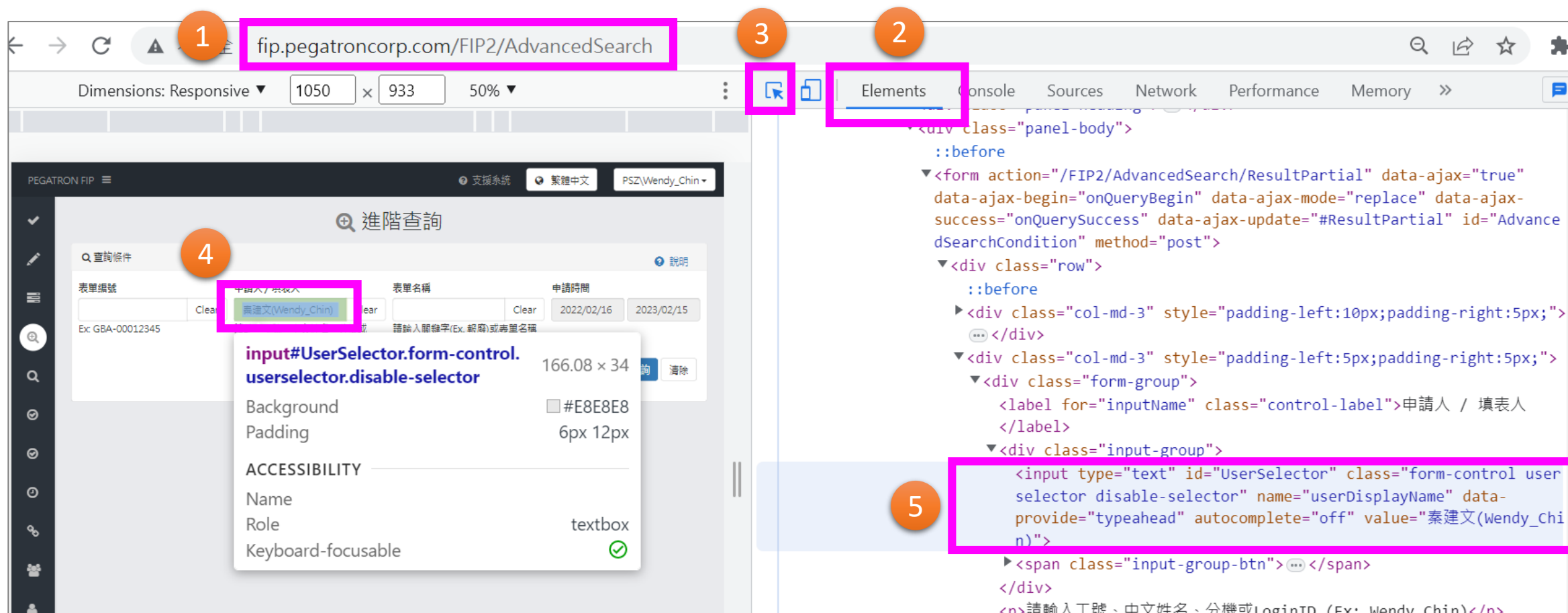
Web:查找目標網頁HTML元素代碼

Step1:打開瀏覽器，輸入目標網頁網址，待網頁加載完后按快捷鍵F12

Step2: 在打開的界面中，選擇Elements

Step3~Step4: 點擊圖示箭頭后，移動鼠標至目標元素處，單擊鼠標左鍵，

Step5: 代碼顏色加深，表示該元素的HTML代碼



Web:Copy目標網頁HTML元素代碼

Step6~Step7: 移動鼠標至加深代碼處，點擊鼠標右鍵顯示快捷菜單，選擇Copy => 繼續選擇Copy的類型，即可。

The screenshot illustrates the process of copying HTML code from a web browser. On the left, a web form titled "進階查詢" (Advanced Search) is visible, featuring input fields for "表單編號" (Form Number), "申請人 / 填表人" (Applicant / Filler), and "申請時間" (Application Time). A tooltip highlights the selected input field with the selector `input#UserSelector.form-control.userselector.disable-selector` and dimensions `166.08 x 34`.

On the right, the browser's developer tools are open, showing the "Elements" panel. The HTML structure is displayed, with the selected input field highlighted in blue. A context menu is open over the selected element, showing various actions. The "Copy" option is selected, and a sub-menu is displayed, showing the following options:

- Copy element
- Copy outerHTML
- Copy selector
- Copy JS path
- Copy styles
- Copy XPath
- Copy full XPath

The steps are numbered 5, 6, and 7, indicating the sequence of actions: 5. Select the element, 6. Click the "Copy" button, and 7. Choose the type of copy to perform.

Selenium範例:定位元素

```
elem1=browser1.find_element(By.ID,'AdvancedSearchMenuLink')
ActionChains(browser1).move_to_element(elem1).click().perform()

elem2=browser1.find_element(By.XPATH,'//*[@id="UserSelector"]')
elem2.send_keys(Keys.CONTROL,'a')
elem2.send_keys(Keys.DELETE)
elem2.send_keys('S09117491')

tday=datetime.date.today()
tdaystr=datetime.date.strftime(tday,'%Y/%m/%d')
date1=tday-datetime.timedelta(days=200)
date1str=datetime.date.strftime(date1,'%Y/%m/%d')

#注意不同瀏覽器readonly有大小寫問題,Chrome支持全小寫readonly
#js代碼獲取方法:元素所在HTML代碼=> 右擊,快捷菜單=>Copy=>Copy JS path
elem3=browser1.find_element(By.XPATH,'//*[@id="StartDate"]')
js='document.querySelector("#StartDate").removeAttribute("readonly")'
browser1.execute_script(js)
elem3.clear()
elem3.send_keys(date1str)
```

Selenium範例:擷取表格元素數據

```
#提取表格數據並轉成pandas dataframe
headdata=[]
tablist=[]
Hder=browser1.find_element(By.XPATH,'//*[@id="SortTable"]').\
    find_elements(By.TAG_NAME,'th')
rows=browser1.find_element(By.XPATH,'//*[@id="SortTable"]').\
    find_elements(By.TAG_NAME,'tr')
for i in Hder:
    headdata.append(i.text)

for rw in rows:
    rwlist=rw.find_elements(By.CSS_SELECTOR,'td')
    datalist = []
    for j in rwlist:
        datalist.append(j.text)
    tablist.append(datalist)

#rows中tr時包括了header, 因此tablist取值時從第二個起
df=pd.DataFrame(tablist[1:],columns=headdata)
print(df)
browser1.close()
```

Selenium範例:下拉菜單、iframe、警示框、指定路徑下載

下拉菜單

```
elem4=browser1.find_element(By.CSS_SELECTOR,'#BPFormView_SelectTypeList')
elem4.click()
Select(elem4).select_by_value('5')
```

iframe

```
#父元素有iframe标签时，需先定位并转换为frame，否则会无法找到子元素
browser1.switch_to.frame('iframeDialog') # 参数为iframe ID
time.sleep(2)
elem8=browser1.find_element(By.CSS_SELECTOR,'#WorkIDTextBox')
elem8.clear()
elem8.send_keys(userID)
```

警示框

```
#警示对话框的处理
time.sleep(2)
alert=browser1.switch_to.alert #获取警示框
alert.accept() #接受警示框，表示“确认”
# alert.dismiss() #解散警示框，表示“取消”
```

指定路徑下載

```
#指定下载路径，并禁止弹出窗口(0表示禁止弹出)
prefs={'profile.default_content_settings.popups':0,'download.default_directory':downpath}
option.add_experimental_option('prefs',prefs)
```

課後作業

1. 在如下的網址中查詢個人一年以來申請的單子, 並將清單導入到Excel中
<http://fip.pegatroncorp.com/FIP2/AdvancedSearch>
2. 自動開立某申請表單, 例如SAT單或者請假單
3. 自動從SFIS 或某個常用網站中下載excel文件

Thank You !